Pedaling Forward

A Scalable and Hybrid Data Architecture for Transport for London

NotebookLM

Team Introduction & Agenda

Agenda

- **Executive Summary & Business Value Propositions -** ***Seok Eng***
- **Technical Solution Design – *Khoon Seng***
- **Data Analysis – *Tashrif, Linglu***
- **Risk & Mitigation, Project Learnings & Retrospective -** ***Edmund***
- **Q&A -** all

Executive Summary

Business Problem: Operational Inefficiency & Cost

Limited visibility into demand patterns leads to bike shortages at high-demand stations, idle assets at low-demand locations, and increased operating cost.

Solution: Scalable Analytics Platform

Build a **scalable, end-to-end** analytics platform that transforms millions of trip records into **automated insights** for **demand forecasting, bike redistribution, and maintenance planning.**

Strategic Impact

- ✔ Enables **predictive decision making**
- ✔ Improves **fleet utilization** (CapEx efficiency)
- ✔ Reduces **rebalancing & maintenance costs** (OpEx savings)
- ✔ Enhances **service reliability** and **customer satisfaction**

Business Value Proposition: Efficiency & Alignment

From Data to Business value

Operational Efficiency

- Demand-driven redistribution reduces manual effort
- Resources deployed where needed most

Cost Optimization

- Predictive maintenance extends fleet lifespan
- Reduced downtime and emergency repairs

Customer Experience

- Higher bike availability
- Consistent and reliable services

Strategic Alignment

- Data foundation for expansion and planning

Problems & Objectives

Key Business Questions We Address

CORE CHALLENGES

- Demand fluctuates by **time, location, and season**
- Network imbalance leads to **empty or full stations**
- Limited visibility into **asset usage and lifecycle**

OUR OBJECTIVES

- Optimize bike distribution across the network
- Reduce shortages during peak demand
- Enable data-driven planning (seasonal & location-based)
- Extend asset lifespan and reduce operational waste

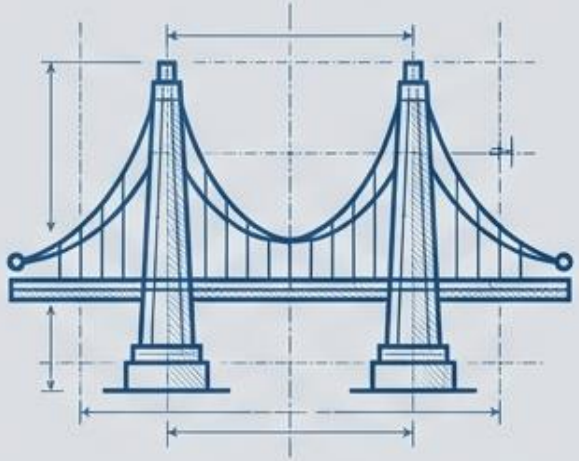
Technical Solution Design

Architecture • Pipeline • Considerations

Resolving the Tension Between Massive Scale and Cloud Economics

The Challenge

Balancing the scalability of massive, highly-granular public datasets with the reality of cloud computing costs and query latency.



The Solution

A Hybrid Data Warehouse architecture combining Google BigQuery's massive storage with local DuckDB's blazing-fast analytics.

The Outcome

Cost-effective, zero-latency, real-time insights into London's cycling network.

NotebookLM

Pedaling Data: A Hybrid Cloud-to-Local Architecture for London Bicycles

EU Region

Cloud Production Tier

BigQuery

GCS Parquet

Cloud-to-Lake Export (EU Region)

Raw data exported from BigQuery to Google Cloud Storage (EU) as compressed Parquet files.

Parquet Files

Local Ingestion Engine

Python-Driven DuckDB Loading

Custom Python scripts load Parquet files from the GCS Data Lake into a local DuckDB instance.

dbt

Shared Transformation Core

dbt Dual-Target Profiles: dbt manages transformation logic across both local DuckDB and production BigQuery warehouse environments.

DuckDB

Local Development Tier

Modeling, Quality & Insights

Analytical Star Schema

Optimized Dimensional Modeling

Raw data transformed into a Star Schema featuring centralized Fact tables and descriptive Dimension tables.

Automated Integrity Testing

Built-in Quality Validation

Automated suites identify nuil values, remove dupiicates, and ensure referential integrity across the model.

Dual Serving Layer

SQLAlchemy

Python & BI Visualization

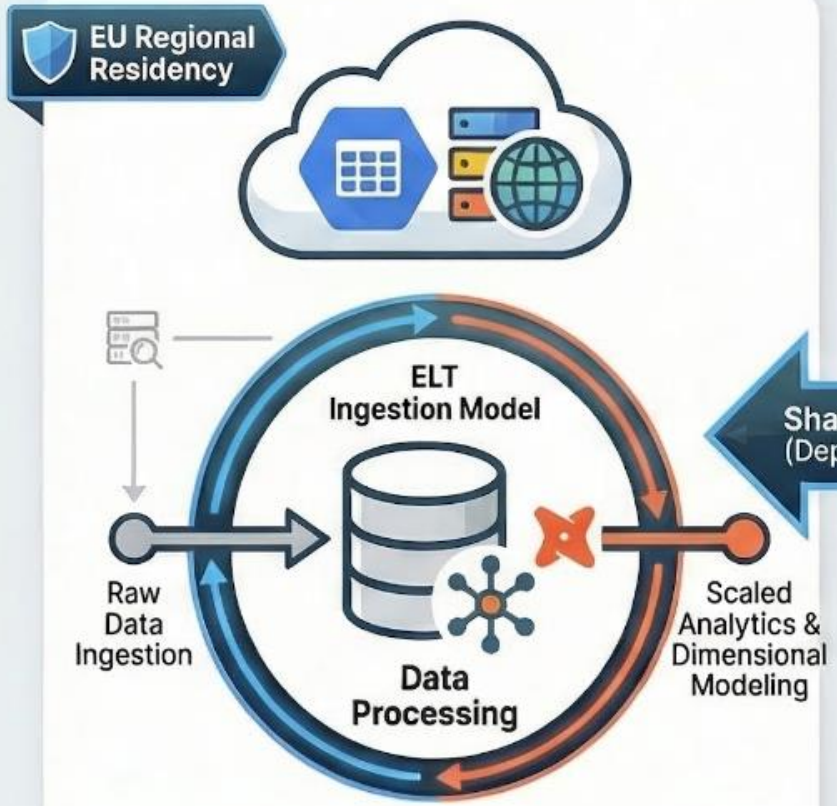
Insights served via SQLAlchemy for Matplotlib/Seaborn charts and professionally integrated with Looker Studio.

Looker Studio

NotebookLM

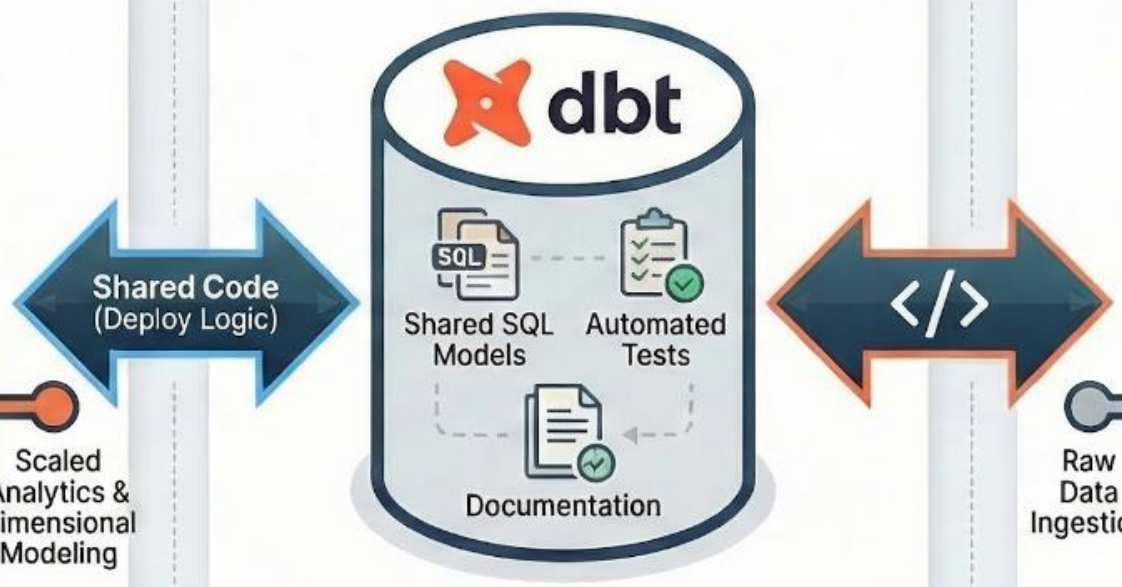
Hybrid Data Transformation: Bridging Cloud Production and Local Development

Cloud Production Tier
(Google BigQuery EU Region)



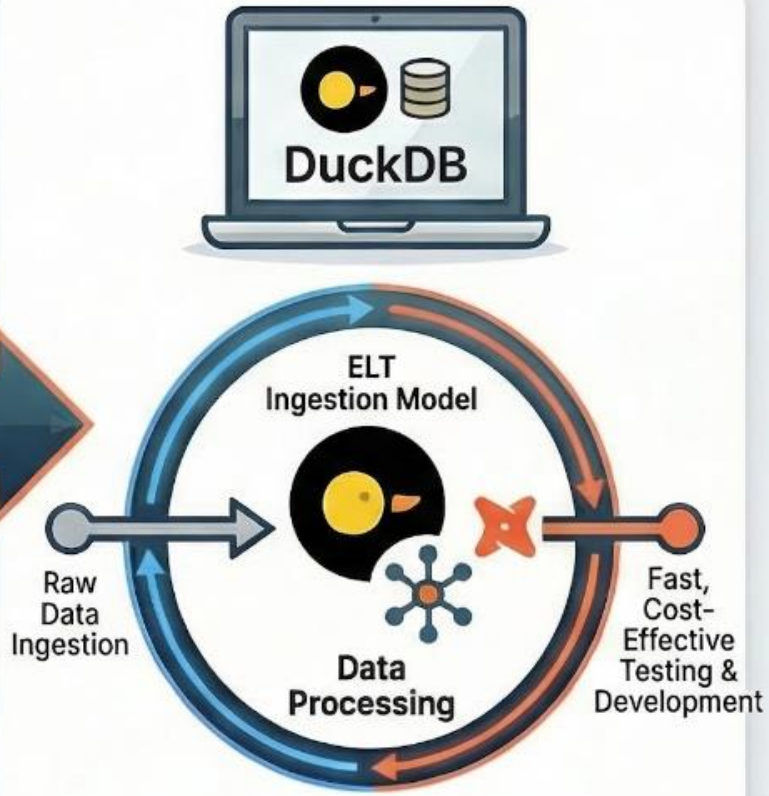
Primary target for scaled analytics, strictly locked to the EU region for residency compliance. Data remains within European geographical boundaries.

The Shared Transformation Core
(dbt)



Central repository hosting shared SQL models, automated tests, and documentation. Version-controlled logic is deployed to both tiers, but **data is never moved between environments**.

Local Development Tier
(DuckDB)



In-process database for developers for fast, cost-effective testing without cloud overhead.

NotebookLM

Data Pipeline Architecture: From Cloud Warehouse to Local Analytics

Step 1: BigQuery (Source)

The process begins at the cloud data warehouse level where the raw 9.57GB dataset resides within Google BigQuery.

Step 2: Export to GCS (Data Lake)

Data is exported from BigQuery into Google Cloud Storage (GCS) to serve as a centralized data lake for processing.

9.57GB Size	Parquet Format Optimization

Files are exported in Parquet format to preserve the original schema and optimize performance for the specific 9.57GB dataset size.

Step 3: Python Ingestion Engine

A custom Python script acts as the primary Ingestion Engine, orchestrating the data movement from the GCS data lake.

Step 4: Local DuckDB (Destination)

The final data is loaded into DuckDB, a local analytical database optimized for read-heavy workloads and complex queries.

NotebookLM

View DBT Documentation

Single dbt Codebase: The DRY Principle in Action

The Central Core: Single Source of Truth

A single, version-controlled directory containing all project configurations and resourees.

dbt_project

SQL Models (Facts & Dimensions)

Modular SQL files that define Fact and Dimension tables for the dimensional model (Star Schema).

Generic Tests & Documentation

Automated data quality checks (e.g., unique, not_null) and searchable documentation that live alongside the code.

The Switch: profiles.yml

Environment Configuration

The profiles.yml file decouples the transformation logic from the connection credentials.

dbt run --target dev

Target: 'dev'

Points to a local DuckDB instance for fast, cost-effective development on the user's machine.

dbt run --target prod

Target: 'prod'

Points to Google BigQuery (EU Region) for the final production data warehouse.

The Result: Strategic Benefits

Adherence to DRY Principle

Eliminates code duplication by using one logic set for all environments.

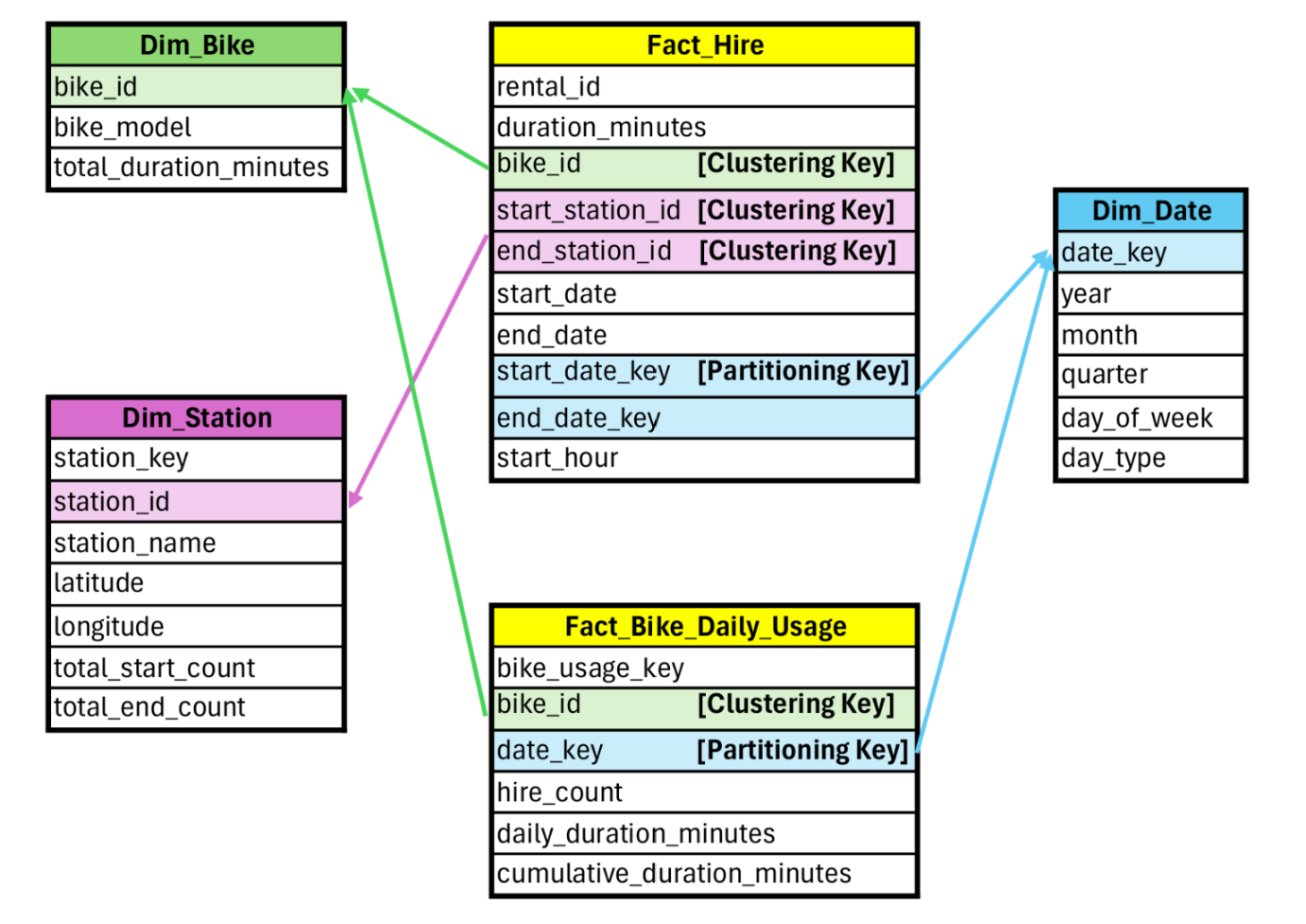
Efficiency & Speed

Local development with DuckDB allows for rapid iteration without cloud latency or costs.

High Data Accuracy

Consistent testing and version control ensure that production models are robust and reliable.

NotebookLM



Dimensional Modeling (Star Schema)

Dimensional Modeling (Star Schema)

Fact_Hire	
rental_id	
duration_minutes	
bike_id	[Clustering Key]
start_station_id	[Clustering Key]
end_station_id	[Clustering Key]
start_date	
end_date	
start_date_key	[Partitioning Key]
end_date_key	
start_hour	

Grain: One row per individual ride
Metric: Usage duration, start hour

Fact_Bike_Daily_Usage	
bike_usage_key	
bike_id	[Clustering Key]
date_key	[Partitioning Key]
hire_count	
daily_duration_minutes	
cumulative_duration_minutes	

Grain: One row per bike per day
Metric: Hire Count, daily usage, lifetime usage of bike over time

Partitioning for Fact Tables

Minimize costs (Cost-Effective)

Maximize query performance (Efficient Querying)

BigQuery Efficiency: Partition Pruning vs. Full Table Scan

In BigQuery's pay-as-you-go model, query costs and speed are determined by the amount of data scanned. Using the **london_bicyeles_dbt** project as a case study, we compare the "Filing Cabinet" analogy to show how partitioning optimizes resource usage.

Full Table Scan:
Searching Every Drawer

10GB vs. Targeted Scanning

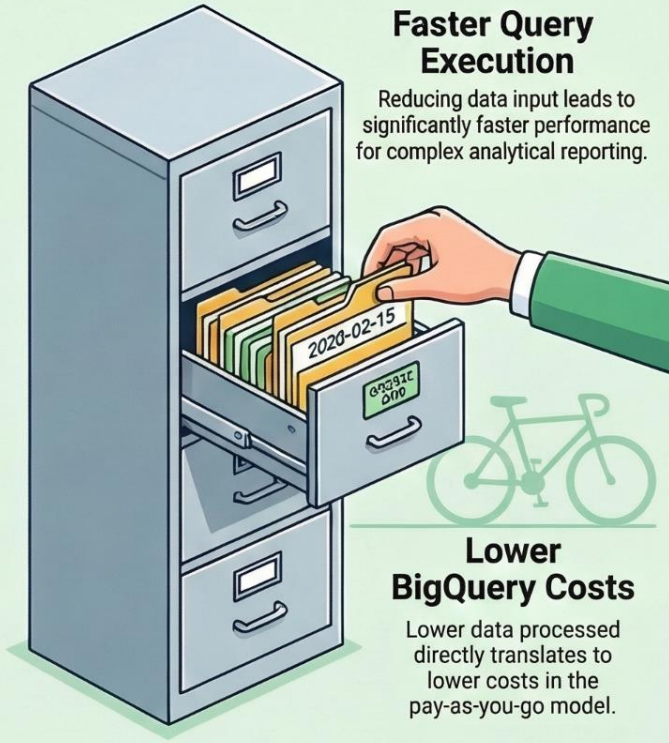
Pruning reduces the scan size from the full 10GB dataset to megabytes.

Data Processed: 10.0 GB

Cost Efficiency: Low (Expensive)

Speed: Slower

Partition Pruning:
Opening the Specific Drawer



Faster Query Execution

Reducing data input leads to significantly faster performance for complex analytical reporting.

Lower BigQuery Costs

Lower data processed directly translates to lower costs in the pay-as-you-go model.

Data Processed: < 500 MB (Example)

Cost Efficiency: High (Cost-Effective)

Speed: Significantly Faster

NotebookLM

Optimizing Data Performance: Clustering the fact_hire Table

Clustering for Fact Tables

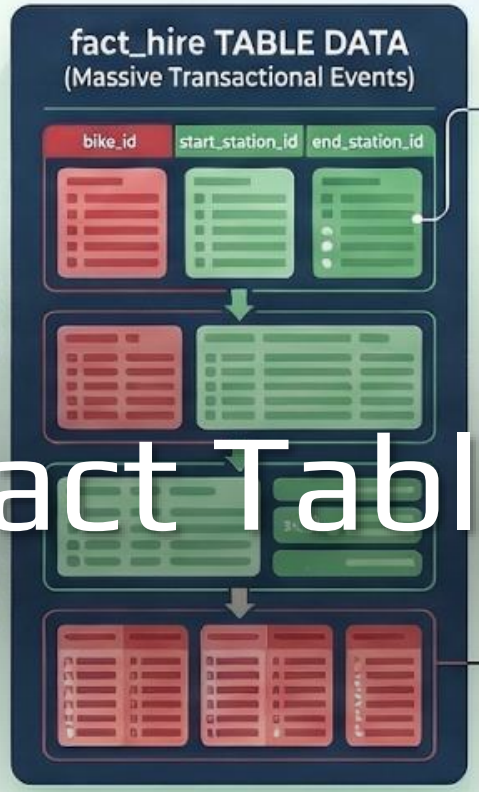
BEFORE CLUSTERING: Full Table Scan

SCAN VOLUME:
100% of Table
(All Data Read)

Query Engine

PROCESS:
Reads every single row to find relevant records. Maximum compute usage.

AFTER CLUSTERING: Intelligent Data Pruning



CHOSEN CLUSTERING KEYS:
Sorting Mechanism

bike_id | start_station_id | end_station_id

Beyond Standard Partitioning: Clustering provides granular micro-sorting within segments for maximum retrieval efficiency.

Query Engine

SCAN VOLUME:
~5% of Table
(Targeted Read)

PROCESS:
Identifies and reads only blocks containing requested IDs. Drastic reduction.

95%
REDUCTION IN DATA SCANNED

50%
REDUCTION IN MAINTENANCE & IT COSTS
(Reduces manual overhead)

PERFORMANCE

QUERY SPEED:
Linear/Slower

EXPENSIVE
(High Compute)

PAY-AS-YOU-GO COST:
High, tied to volume scanned.

METRIC	BEFORE CLUSTERING	AFTER CLUSTERING
EXECUTION METHOD	Full Table Scan	Intelligent Data Pruning
SCAN VOLUME	100% of Table	~5% of Table
QUERY SPEED	Linear/Slower	Near-Instant/Optimized
COST EFFICIENCY	Expensive	Highly Efficient

PERFORMANCE

HIGHLY EFFICIENT
(Low Compute)

FINANCIAL IMPACT:
Directly lowers monthly bill.

SUSTAINABLE ANALYTICS AT SCALE:
Stable performance for billions of rowe.

NotebookLM

Query Cost Reduction With DB Optimization

Query Example 1

Compare rental count for specific months between classic and e-bikes

==>

Query Cost Reduced from 1.87GB to 62MB

30x improvement

FACT_HIRE

```sql
-- Compare rental count for specific months between classic and ebikes
SELECT
    DD.year,
    DD.month,
    DB.bike_model,
    COUNT(FH.rental_id) AS total_rentals
FROM
    `london-bike-unoptimized.london_bicycles_star.fact_hire`  AS FH
INNER JOIN
    `london-bike-unoptimized.london_bicycles_star.dim_date`  AS DD ON FH.start_date_key = DD.date_key
INNER JOIN
    `london-bike-unoptimized.london_bicycles_star.dim_bike`  AS DB ON FH.bike_id = DB.bike_id
WHERE start_date_key between '2022-09-01' and '2023-01-31'
GROUP BY
    year, month, bike_model
ORDER BY
    year, month, bike_model
```

This query will process 1.87 GB when run.

```sql
-- Compare rental count for specific months between classic and ebikes
SELECT
    DD.year,
    DD.month,
    DB.bike_model,
    COUNT(FH.rental_id) AS total_rentals
FROM
    `testgcp-487607.london_bicycles_star.fact_hire`  AS FH
INNER JOIN
    `testgcp-487607.london_bicycles_star.dim_date`  AS DD ON FH.start_date_key = DD.date_key
INNER JOIN
    `testgcp-487607.london_bicycles_star.dim_bike`  AS DB ON FH.bike_id = DB.bike_id
WHERE start_date_key between '2022-09-01' and '2023-01-31'
GROUP BY
    year, month, bike_model
ORDER BY
    year, month, bike_model
```

This query will process 62.23 MB when run.

Query Cost Reduction With DB Optimization

Query Example 2

Find bikes that were hired more than 180 times in a month

==>

Query Cost Reduced from 2.11GB to 90MB

23x improvement

FACT_BIKE_DAILY_USAGE

```
-- Find bikes that were hired more than 180 times in a month
SELECT
    DD.year,
    DD.month,
    DU.bike_id,
    sum(DU.hire_count) as TotalRentals
FROM `london-bike-unoptimized.london_bicycles_star.fact_bike_daily_usage` DU
INNER JOIN
    `london-bike-unoptimized.london_bicycles_star.dim_date` DD on DD.date_key = DU.date_key
INNER JOIN
    `london-bike-unoptimized.london_bicycles_star.dim_bike` DB on DU.bike_id = DB.bike_id
WHERE DU.date_key between '2022-10-01' and '2023-01-31'
group by
    DD.year, DD.month, DU.bike_id
having TotalRentals > 180
order by
    DD.year, DD.month, TotalRentals desc, DU.bike_id
```

This query will process 2.11 GB when run.

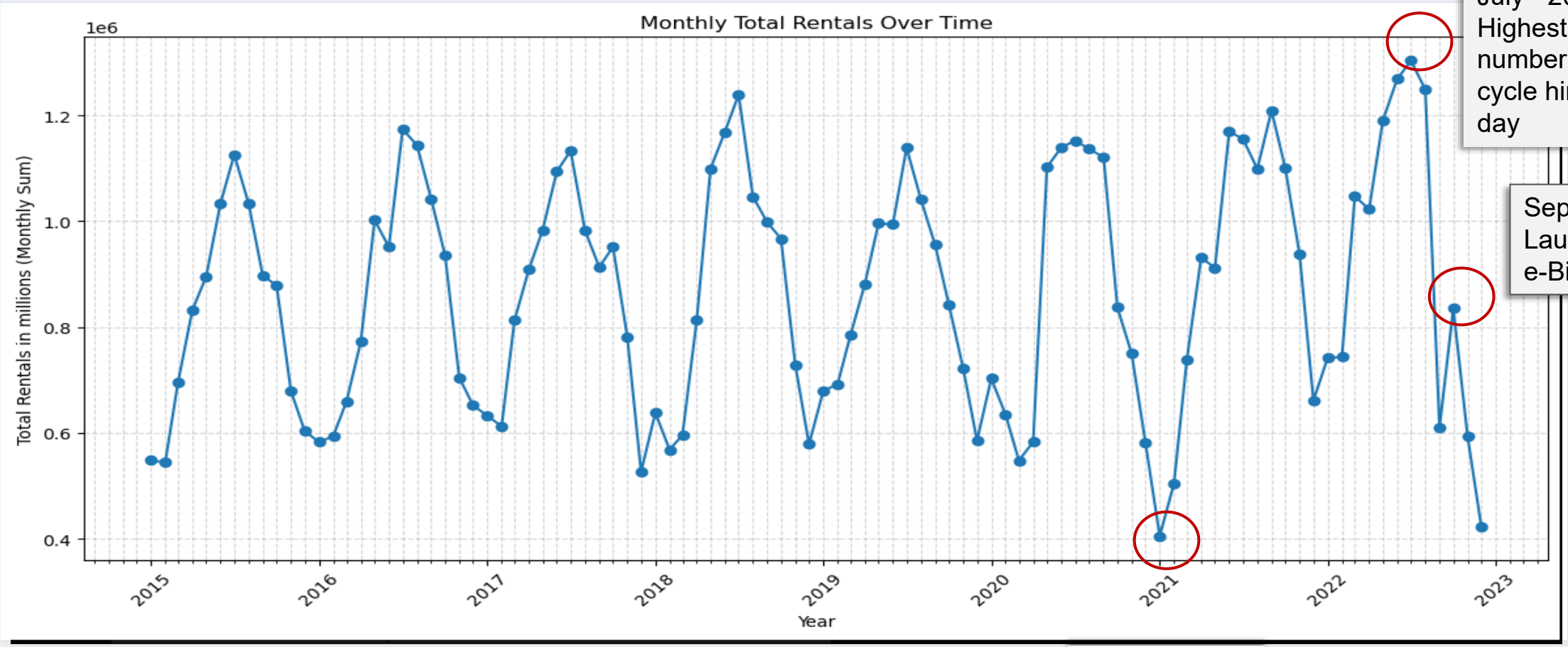
```
-- Find bikes that were hired more than 180 times in a month
SELECT
    DD.year,
    DD.month,
    DU.bike_id,
    sum(DU.hire_count) as TotalRentals
FROM `testgcp-487607.london_bicycles_star.fact_bike_daily_usage` DU
INNER JOIN
    `testgcp-487607.london_bicycles_star.dim_date` DD on DD.date_key = DU.date_key
INNER JOIN
    `testgcp-487607.london_bicycles_star.dim_bike` DB on DU.bike_id = DB.bike_id
WHERE DU.date_key between '2022-10-01' and '2023-01-31'
group by
    DD.year, DD.month, DU.bike_id
having TotalRentals > 180
order by
    DD.year, DD.month, TotalRentals desc, DU.bike_id
```

This query will process 90.18 MB when run.

Data Analysis

Demand Trends & Patterns

Demand is predictable: seasonality + weekly rhythm



July - 2022
Highest number of cycle hire in a day

Sep - 2022
Launch of e-Bike

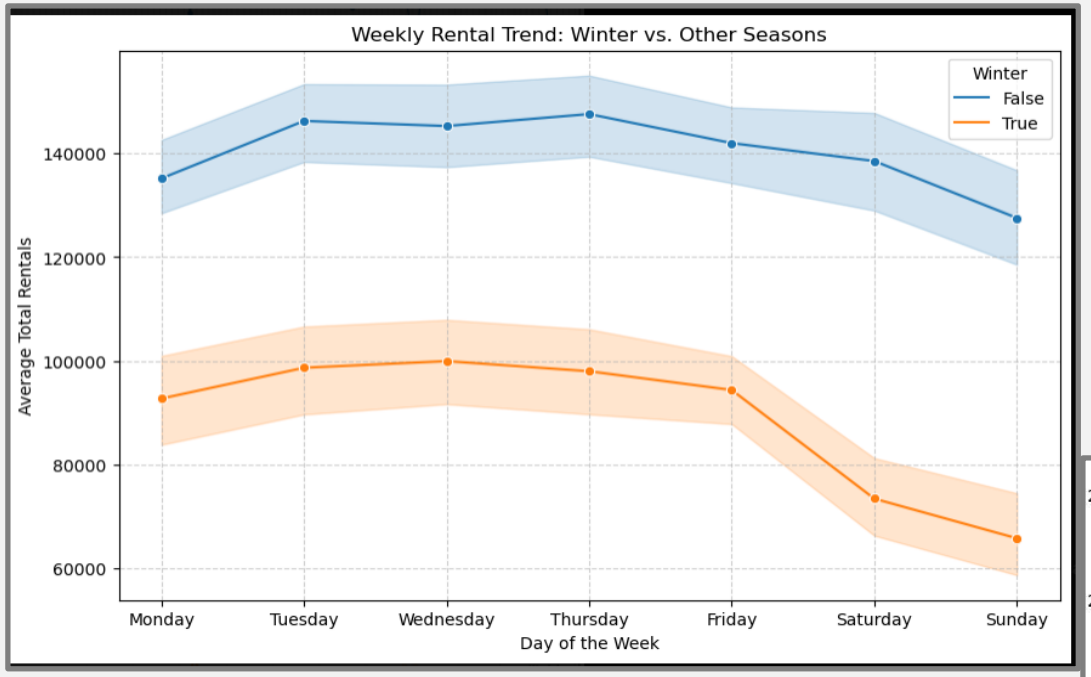
Jan - 2021
Covid Lockdown

Key Insights

Strong seasonal variation (> 50%)

Demand Trends & Patterns

Demand is predictable: seasonality + weekly rhythm



Key Insights

Weekdays: commuter peaks (AM & PM)

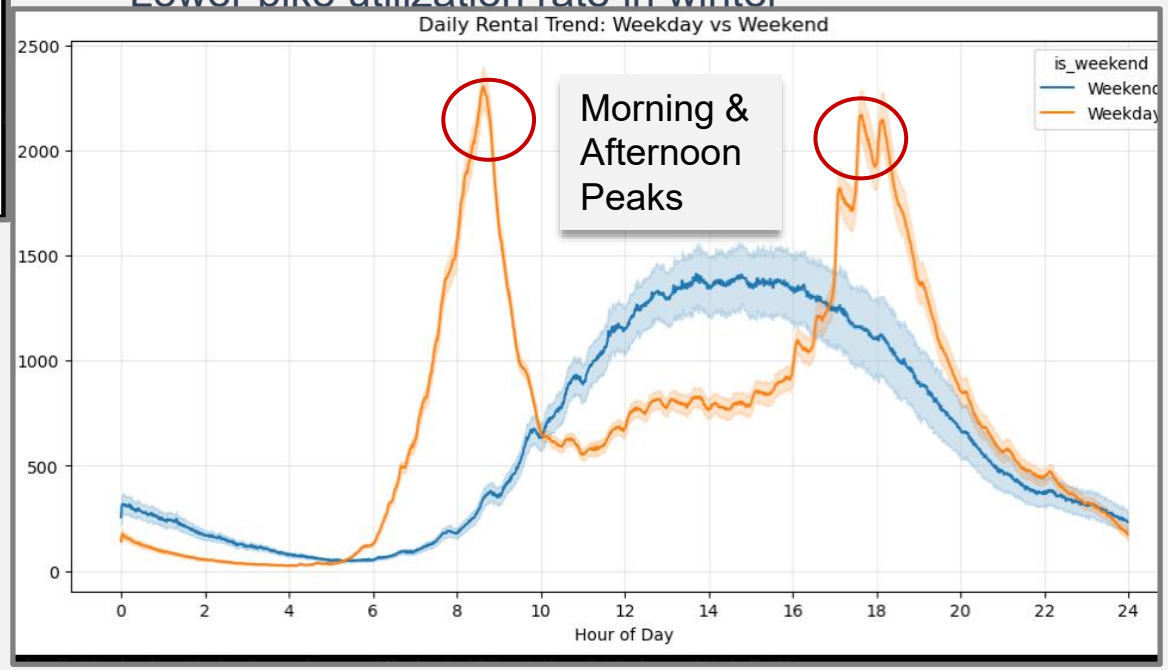
Weekends: midday leisure usage

Strong seasonal variation (~ 40%)

Business Implication

Demand can be forecasted and optimized

Lower bike utilization rate in winter



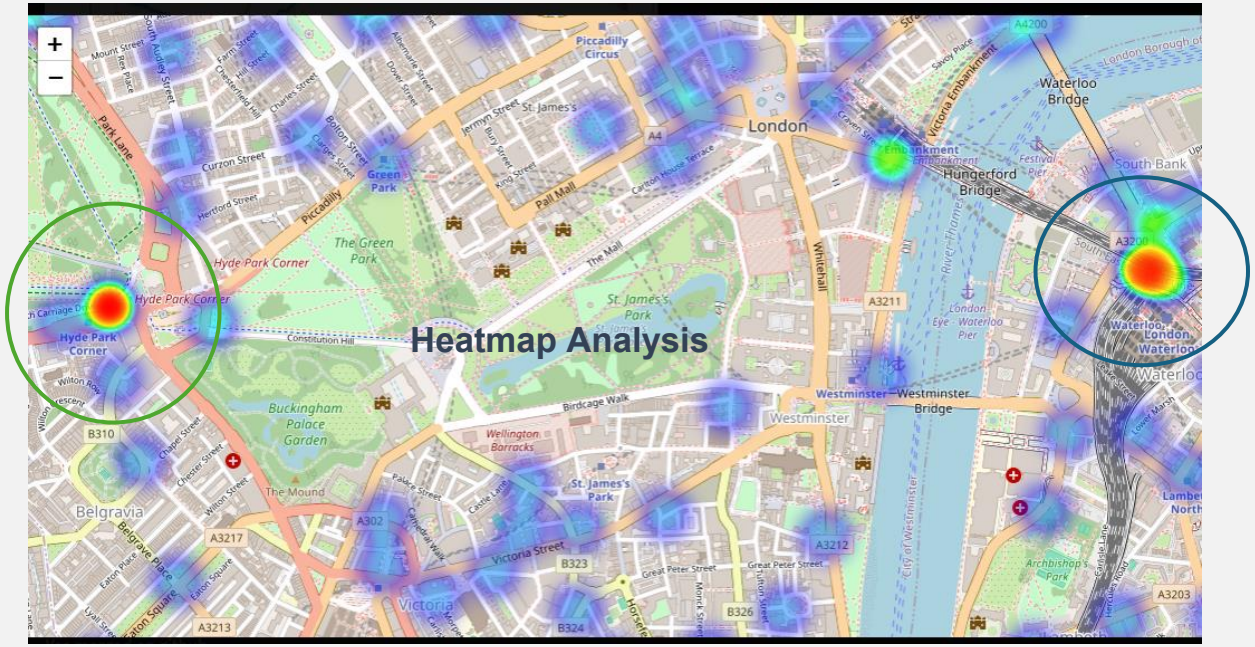
Actions

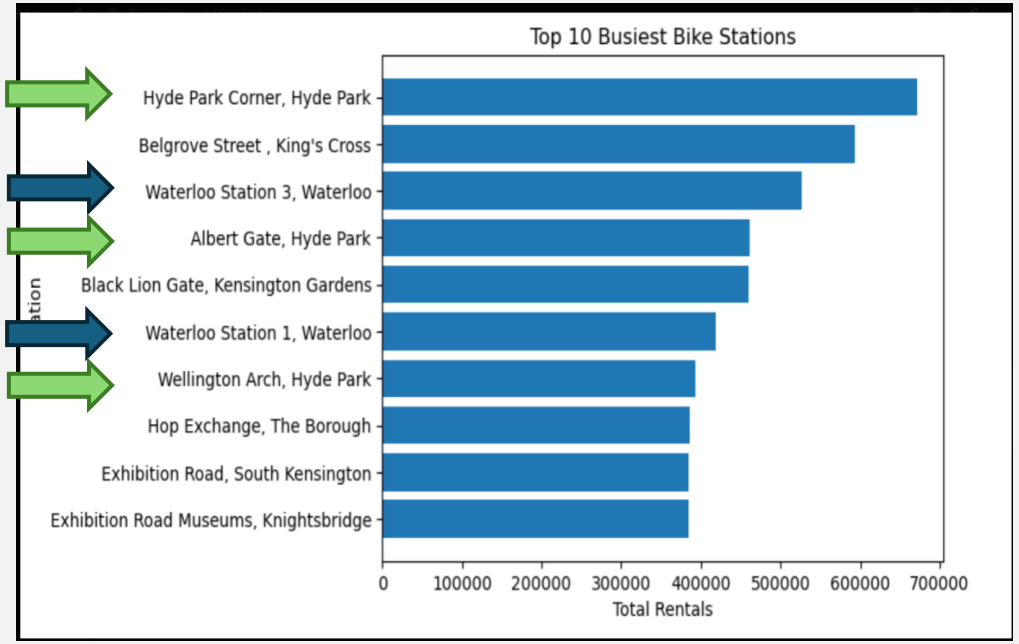
Align supply with time-based demand

Schedule maintenance during off-peak periods

Station-Level Insights

Demand is concentrated in Key Hubs.





Key Insights:

A small number of stations drive disproportionate demand

Business Impact:

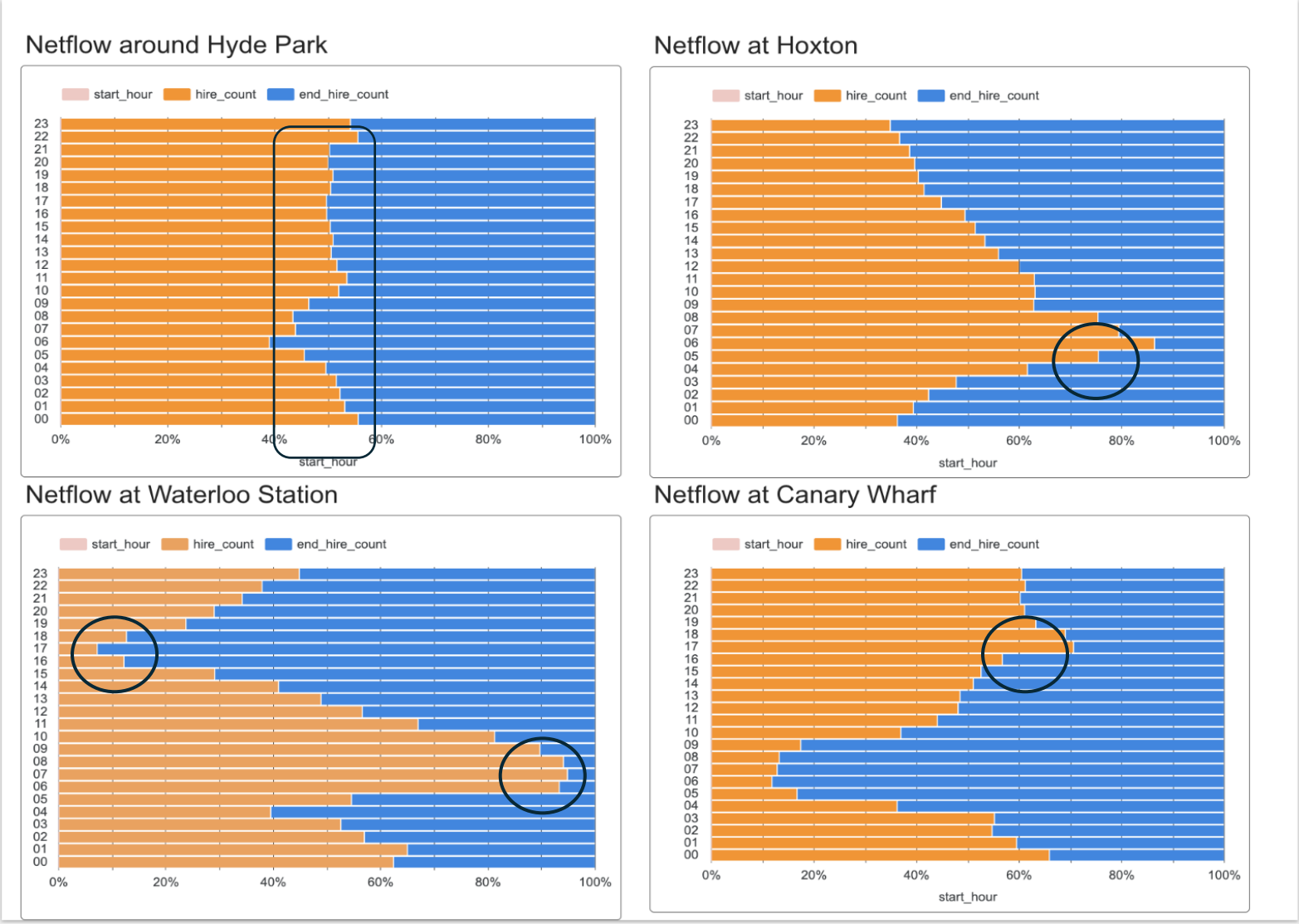
High risk of shortages at critical locations

Actions:

Prioritize high-demand hubs
Optimize rebalancing using flow patterns
Expand docking capacity where needed

Station-level Insights: Netflow

Business Value: Optimize redistribution to reduce manual rebalancing costs.



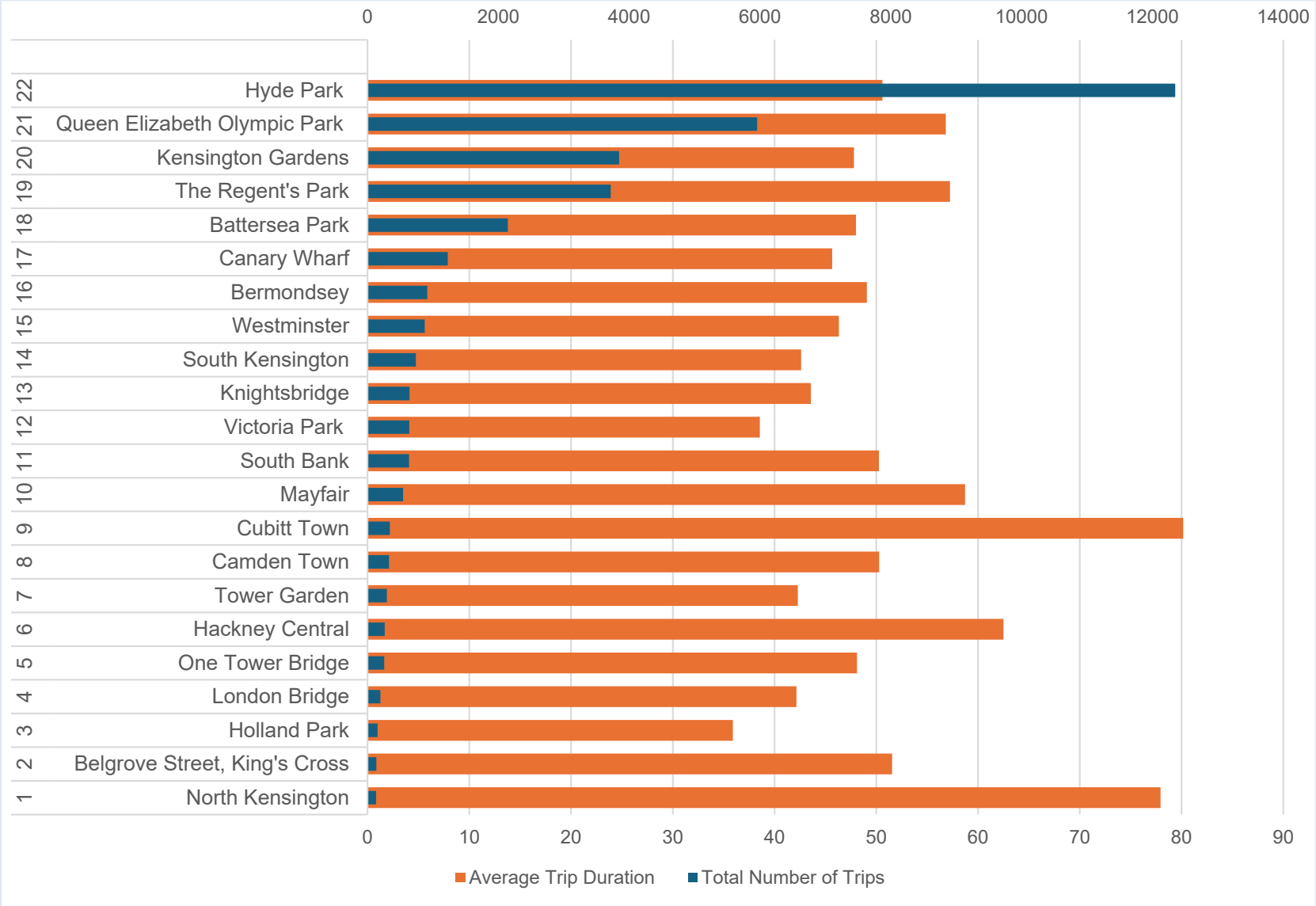
Key Findings

- clusters show different usage patterns reflecting different user profiles
- These patterns create systematic imbalance across the network

Actions:

- Prioritize high-demand hubs
- Optimize rebalancing using flow patterns
- Expand docking capacity where needed
- live monitoring dashboard for supporting demand surge at self-sustaining clusters

Top 20 Most Popular Round-Trip Routes



Key Insights

- Round the park trips are popular, even if it takes longer time to complete

Business Impact

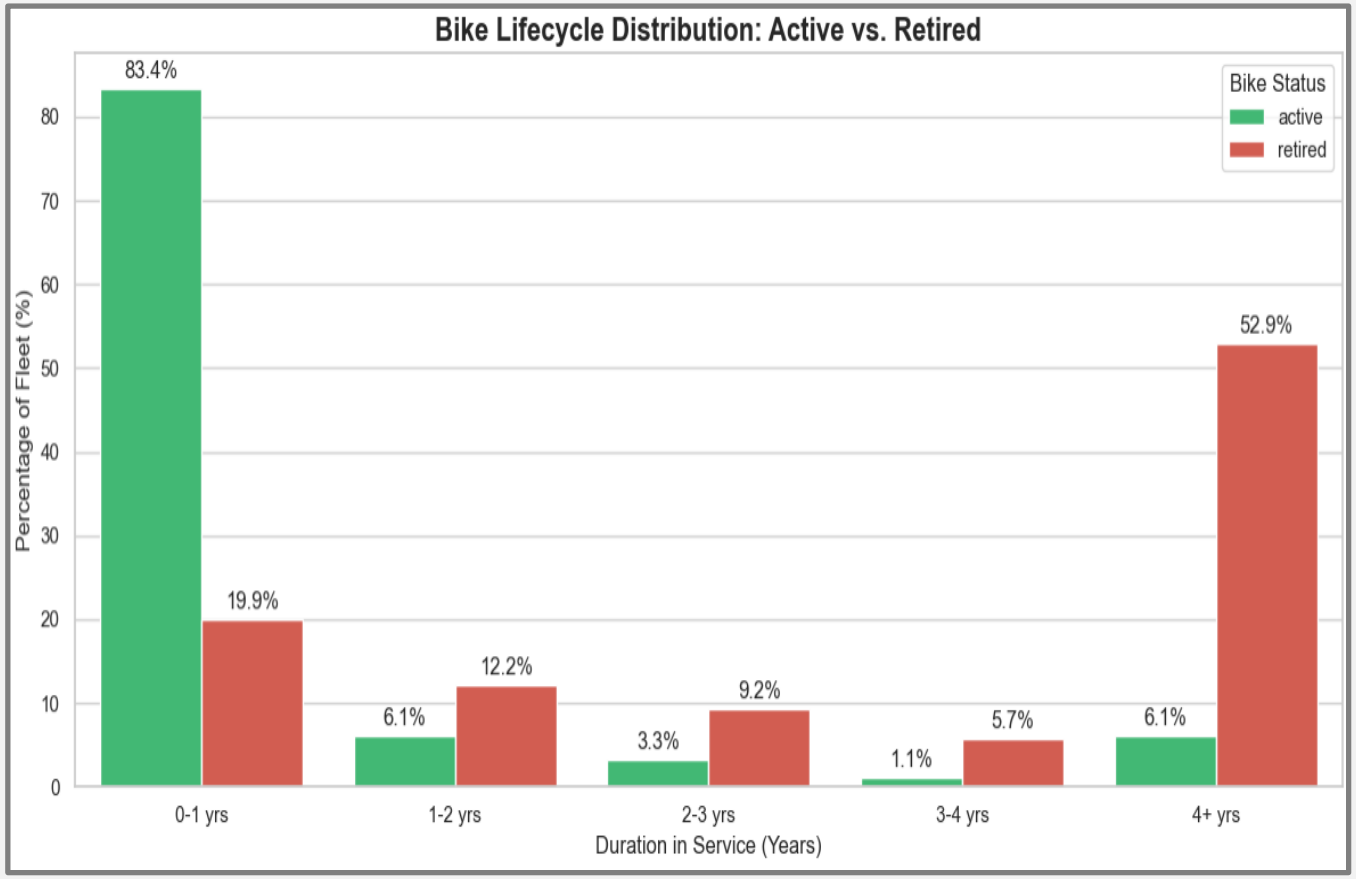
- Different demand profiles
- different operational needs

Actions

- Prioritize supply along commuter routes
- Introduce pricing/promotions for leisure usage

Asset Lifecycle Insights

Proactive Maintenance Drives Cost Efficiency



Key Insights

- Bike lifecycle is predictable and data-driven
- >80% of bikes are less than 1 year old.
- Typical lifespan of a classic bike is 4 years

Business Impact

- Reduce downtime
- Optimize capital replacement planning

Actions:

- Implement preventive maintenance
- Plan phased fleet replacement

Recommendations & Roadmap

Actionable steps to improve efficiency, availability, and user experience.

Short-Term

Improve redistribution at key hubs

Focus on peak demand management

Mid-Term

Deploy predictive models

Integrate external data (weather, events)

Long-Term

Enable smart city mobility planning

Scale to new regions/ services

Risk and Mitigation

Risk Assessment & Mitigation Strategy

Addressing potential limitations to ensure operational resilience.

Risk: Data Latency
Delayed station updates could lead to "stale" or incorrect redistribution of bikes
Mitigation: Implement streaming IoT ingestion (15s latency) vs batch, to ensure the bikes rides are updated

Risk: Extreme Seasonality
Models trained on summer data may fail in winter.
Mitigation: Use seasonally adjusted baseline models and weather APIs or separate models for projections or forecasting

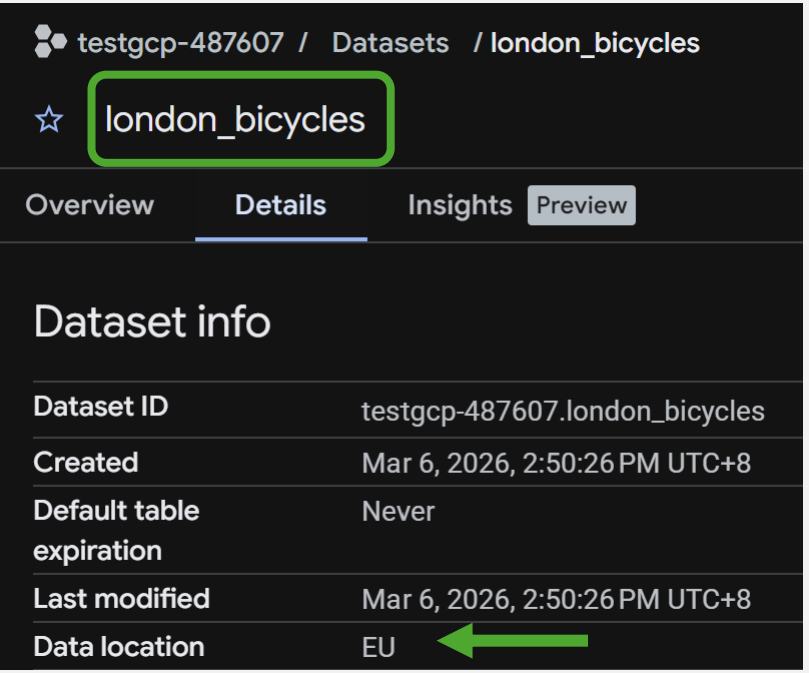
Risk: Hardware and Software Limits
a.Physical dock capacity is fixed however algorithms may predict more bikes needed during peak hours
b.Technical Issues (Cloud quota, technical complexity)
Mitigation: a.Trigger "overflow protocols" (valet service i.e. extra bikes on transport vehicles) to cater for super-peaks.
b.Use of datalake in google cloud storage to store parquet files and optimization of code

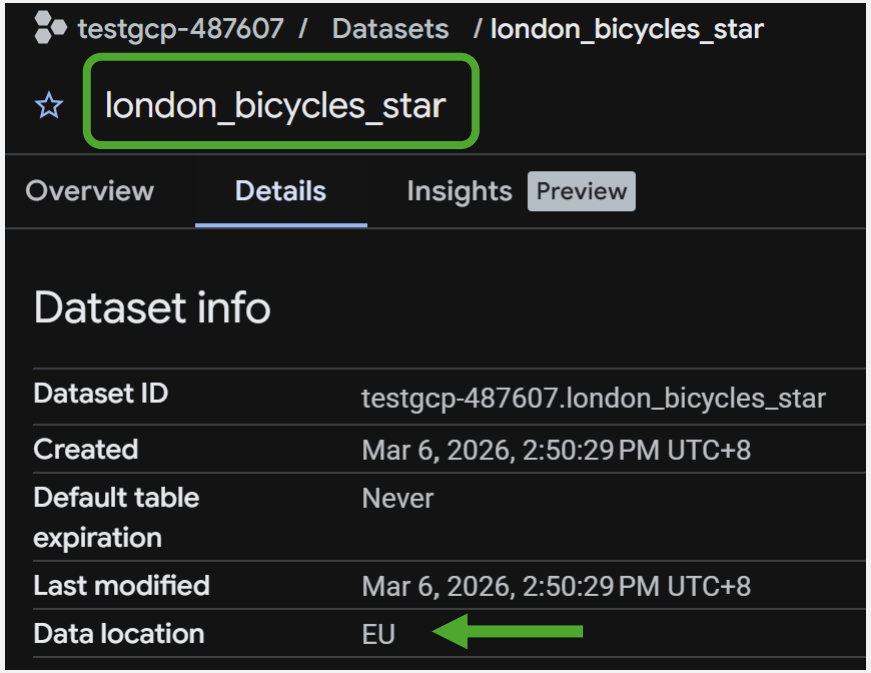
Risk: GDPR/Data Privacy/Regulations
Handling personal user trip data requires strict protection and prevention of data leakage.
Mitigation: Anonymize all personal trip data when sending out reports; limit access to privileged accounts only

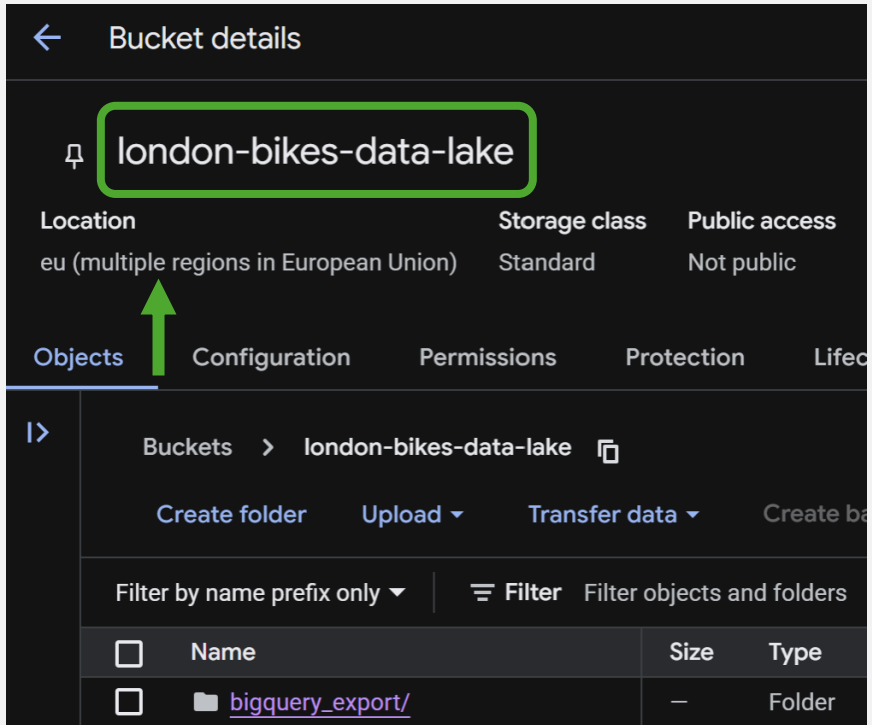
.

Anticipating Technical Risks and Limitations

Risk Profile 1	Mitigation Strategy
Regional Data Residency and GDPR Compliance	All infrastructures (including GCS buckets and BigQuery datasets) is pinned to the **EU region**.



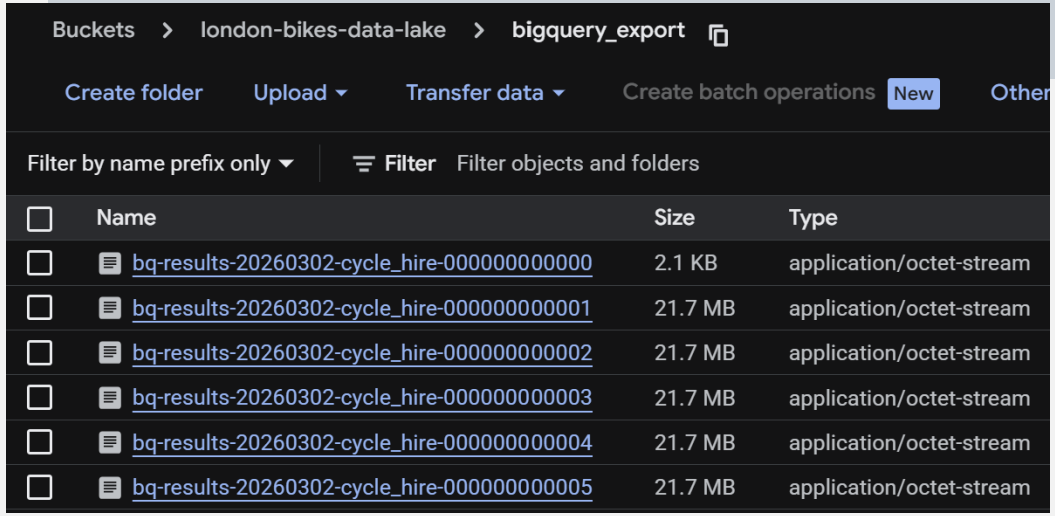




Anticipating Technical Risks and Limitations

Risk Profile 2

Cloud Quota Overruns and Unpredictable Costs

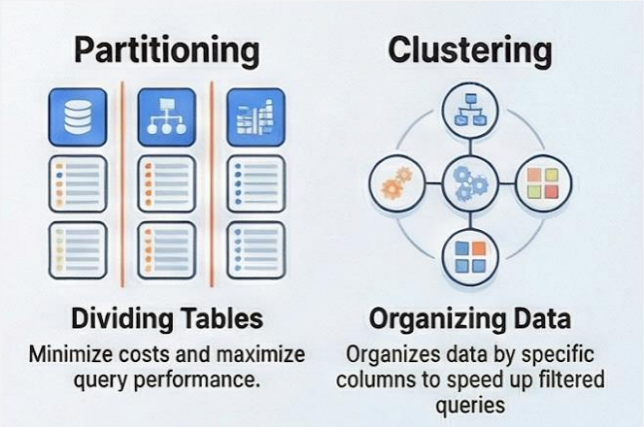


Mitigation Strategy

The hybrid architecture **minimizes** active BigQuery **compute time**. Heavy analytical processing can be tested locally, reducing the risk of incurring query costs.

Data Lake on Google Cloud Storage stores extracted parquet files which utilizes advanced compression algorithms (e.g. Snappy) to significantly reduce storage costs and are optimized for high-performance queries.

Implement cost optimization techniques via partitioning and clustering.



Anticipating Technical Risks and Limitations

Risk Profile 3	Mitigation Strategy
Technical Complexity and Maintainability with single code base	Always use common SQL functions supported by both DuckDB and BigQuery to minimize usage of Jinja If no common SQL functions, use macros to create functions to return database-specific syntax based on target environment Enforce Dual Environment Testing for every change

generate_date_key.sql M

london_bicycles_dbt > macros > generate_date_key.sql

```
{% macro generate_date_key(column_name) %}
    {# Detect the database type and return the correct formatting function #}
    {% if target.name == 'cloud' %}
        DATE({{ column_name }})
    {% elif target.name == 'local' %}
        CAST({{ column_name }} as DATE)
    {% else %}
        {{ column_name }} {# Fallback if target is neither #}
    {% endif %}
{% endmacro %}
```

Anticipating Technical Risks and Limitations

Risk Profile 4	Mitigation Strategy
Missing or Incomplete Data, Data inconsistency Examples: -Missing station IDs -Null start/end timestamps -Trips without bike identifiers **Impact**: Incorrect trip counts Wrong demand forecasting Distorted usage patterns These risks affect the accuracy and reliability of London bicycles database.	Input 0 for records without station ID, bike identifiers Segregate data for classic and e-bikes for business insights

Operational Risk

Risk Profile 5	Mitigation Strategy
Station Imbalance Risk -Common issue in bike sharing systems: -Bikes accumulate in city center in morning -Suburban stations become empty **Impact**: User dissatisfaction Reduced service availability These arise from the physical bike sharing system	Bike-sharing systems frequently experience stations being empty or full depending on commuting patterns, requiring constant rebalancing operations.

Cybersecurity and Data Privacy Risks

Risk Profile 6	Mitigation Strategy
Re-identification Risk Trip patterns can identify individuals. **Impact**: Privacy breach GDPR violations **API or Data Pipeline Vulnerabilities** **Potential risks:** Unauthorized database access Data tampering Fake trip injection **Impact**: Wrong analytics Operational disruption	Focus on data preventative and detective controls and vulnerabilities assessment and periodic control monitoring

Strategic / Policy Risks

Risk Profile 7	Mitigation Strategy
Transport Policy Changes Examples: New cycling infrastructure Congestion charges Public transport pricing **Impact**: Demand changes. **Public Health Events** **Example**: COVID-19 significantly altered bike usage patterns before recovering. **Impact**: Demand shocks.	Review of existing policies, new policies and perform gap analysis of the compliance against the policies

Project Learnings & Retrospective

Key takeaways from the development and implementation phase.

Data Quality First

Investing time in upstream validation rules prevented 80% of downstream reporting errors. Data quality drives downstream success.

Operational Context Matters

Optional context is critical for actionable insights.

Iterative Deployment

Pilot => scale improves outcomes.

"This project demonstrates how modern data engineering tools can transform raw transportation data into actionable insights that support smarter urban mobility planning."

Q&A Session